Proiect “Programare paralela pentru Unitatea de Procesare Grafica”

- 3D MMO Virtual World Physics folosind GPGPU –

Proiectul se poate realiza in echipe de maxim 2 studenți.

A. Prototip 1 si 2 - 3D Virtual World Physics

Se vor studia noțiuni despre operațiile de fizica efectuate in cadrul simulărilor de lumi virtuale 3D.

Noțiuni minimale de studiat:

- Bounding Volumes: teorie, avantaje, formule, etc
  - AABB : Axis Aligned Bounding Boxes
  - OBB : Object Oriented Bounding Boxes
  - etc.
- Collision detection:
  - Intersecții de tipul Bounding Volumes / Ray to Triangles : teorie, avantaje, formule, etc.
- Forte implicate in simulările fizice pentru creșterea realismului:
  - Friction
  - Gravity
  - etc.

Folosind noțiunile studiate, se va implementa următorul program OpenGL / Calcule de fizica cu **următoarele funcționalități**:

- Collision intre obiecte (collision detection si collision response)
- Friction
- Gravity

Vor exista **următoarele tipuri de obiecte** :

- Sfera / Calcule de fizica folosind Bounding Sphere
- Cub / Calcule de fizica folosind Bounding Volume
- Cilindru sau Con / Calcule de fizica folosind mesh-ul obiectului generat (Ray To Triangles / Triangles To Triangles) SAU se poate aproxima geometria pentru coliziune calculând un volum încadrator

Obiectele vor fi plasate si se vor mișca in interiorul unei "cutii" (box).

Acest box trebuie sa aibă fata prin care privește utilizatorul 'invizibila' (logic ea se afla in scena) astfel încât sa se poată vedea obiectele din interior.

Poziția inițiala a unui obiect generat, viteza si direcția se vor determina aleator.

Vor exista 2 moduri de funcționare :

1. Programul trebuie sa permită utilizatorului sa genereze oricând dorește tipuri de obiecte in interiorul cutiei. De asemenea, utilizatorul trebuie sa aibă posibilitatea sa reseteze / imprime mișcare obiectelor aflate in cutie astfel încât daca la un moment dat in urma mișcării obiectelor acestea s-au 'așezat' si nu se mai deplasează, scena sa devina din nou dinamica.
2. Modul cu obiecte generate predefinit, in care toate obiectele din scena vor fi generate automat la începutul simulării. Vor exista 3 moduri predefinite astfel :
    - 100 de obiecte de tip sfera ; 250 de obiecte de tip cub; 500 de obiecte de tip cilindru / con
    - 250 de obiecte de tip sfera ; 500 de obiecte de tip cub; 1000 de obiecte de tip cilindru / con
    - 500 de obiecte de tip sfera ; 1000 de obiecte de tip cub; 2500 de obiecte de tip cilindru / con

Ultima funcționalitate care trebuie implementata este afișarea FPS-ului din scena / calculul cat durează procesarea operațiilor de fizica astfel încât pentru fiecare dintre cele 3 moduri predefinite sa se poate vedea viteza cu care funcționează.

Vor exista 2 prototipuri / implementări diferite din punct de vedere al calculelor de fizica :

- **Prototip 1 - 3D Virtual World Physics**: o implementare in care operațiile de fizica se vor executa pe CPU
- **Prototip 2 - 3D Virtual World Physics**: o implementare in care operațiile de fizica se vor executa folosind prelucrări GPGPU

Se dorește astfel realizarea unei comparații intre cele doua implementări cu evidențierea avantajelor/dezavantajelor existente.

OBSERVATII :

Pentru generarea mesh-ului exista 2 variante:

1. Fie se calculează programatic geometria (quads / triangles) din care este alcătuit obiectul respectiv, folosind formule matematice.
2. Fie se încarcă obiectele respective folosind un format de fișier 3D simplu (cum ar fi de exemplu formatul .off : http://shape.cs.princeton.edu/benchmark/documentation/off_format.html). Atenție, ca sa existe iluminare corecta trebuie sa fie calculate si normalele in vârfurile poligoanelor geometriei (quads / triangles). Daca formatul de fișier folosit nu are precalculate si normalele, acestea se calculează folosind produsul vectorial (exemplu : https://www.khronos.org/opengl/wiki/Calculating_a_Surface_Normal) si apoi se efectuează normalizarea rezultatului.
3. Se poate folosi pentru redarea / construirea scenei 3D framework-ul de la PPBG

B. Prototip 3 - 3D MMO Virtual World Physics

Realizarea unui prototip funcțional pentru un spațiu virtual 3D MMO. Astfel se dorește realizarea celor doua componente, client si server, unde obiectivul principal este ca modulul de server sa folosească pentru procesarea taskurilor de calcul intensive tehnici GPGPU.

MODULUL SERVER

In arhitectura serverului trebuie sa existe cel puțin următoarele module :

- **Alocare clienți**: modul ce este responsabil cu alocarea clienților pe resursele disponibile pentru procesare
- **Scheduler**: acest modul este responsabil cu planificarea taskurilor; vor exista pentru a putea vedea diferențele de performanta doua submodule :
    - GPU scheduler
    - CPU scheduler
- Se va putea seta in parametrii serverului care dintre cele doua este folosit.
- Procesare : acest modul este responsabil cu execuția propriu zisa a taskurilor pe server; similar cu modulul de scheduler vor exista si aici doua submodule :
    - Procesare GPU
    - Procesare CPU

Serverul trebuie sa ofere cel puțin următoarele funcționalități către clienții ce accesează spațiul virtual:

- Permitere conectare clienți
- Autentificare clienți
- Trimitere evenimente către clienți:
    - Update al poziției acestora
    - Update-uri legate de setările lumii virtuale (de exemplu cu cat se poate deplasa la fiecare iterație in lumea virtuala)
    - Update al altor utilizatori conectați in spațiul virtual si care se afla in aceeași zona cu utilizatorul care primește update-ul
    - Update-uri legate de stări ale lumii virtuale
    - Update-uri legate de geografia lumii virtuale
    - Update-uri privind mesaje trimise de utilizatori intre ei
    - Etc.
- Verificare corectitudine deplasare utilizator in cadrul spațiului virtual
- Utilizatorii vor trebui sa se deplaseze corect respectând următoarele restricții:
    - Nu vor putea sa „intre” intr-un alt utilizator
    - Nu vor putea sa „intre” intr-un alt obiect din spațiul virtual
    - Nu vor putea sa „zboare”
    - Nu vor putea sa ajungă sub teren

    - Nu se vor putea deplasa decât cu o viteza prestabilita; cu alte cuvinte daca un utilizator va încerca sa se deplaseze de la coordonatele curente la niște coordonate unde pentru a ajunge depășește viteza de deplasare impusa de spațiul virtual, mișcarea nu va fi considerata valida
    - Etc.

- Bucla principala a simulării lumii virtuale:
    - Nucleul unei aplicații MMO este reprezentat de o bucla de simulare care se executa sincronizat cu un framerate; pe parcursul fiecărei iterații (tick) ale buclei de simulare, porțiuni ale stării sunt actualizate in concordanta cu logica simulării spațiului virtual; se poate considera pentru sincronizare un framerate de 30 (iterație/tick a buclei la aproximativ fiecare 33 milisecunde) sau 60 (iterație/tick a buclei la aproximativ fiecare 16 milisecunde)
- Descrierea geografiei lumii virtuale:
    - Pentru simplitate se poate folosi un teren planar ce conține obstacole simple (sfere, cuburi, cilindri, etc)
    - Se poate decide ca aceasta geografie sa fie „partajata” cu aplicația client : practic exista o configurație default (echivalentul unui fișier harta) pe care aplicația client li are default; la prima conectare si la conectările ulterioare ale clientului la spațiul virtual se pot primi update-uri ale acestei geografii astfel încât clientul își modifica corespunzător „harta”

Serverul trebuie sa pună la dispoziție cel puțin următoarele funcții de API pentru clienți :

- **Login()**: Conectare si autentificare la spațiul virtual
- **GetPosition()**: se obține poziția clientului in spațiul virtual; aceasta poziție va fi folosita atunci când utilizatorul accesează de fiecare data spațiul virtual pentru a fi „poziționat” unde trebuie deoarece el poate părăsi spațiul virtual când se afla in locuri diferite ale acestuia (atenție: astfel este necesar ca serverul sa stocheze si pe disc / DB pozițiile utilizatorilor astfel încât sa poată sa le furnizeze si daca serverul este oprit / pornit din nou)
- **SetPosition()**: setează poziții pentru clienți ; aceasta funcție se va folosi in mod special de către server pentru a trimite unui utilizator al spațiului virtual, pozițiile celorlalți utilizatori (fie când aceștia se conectează, se deplasează, etc)
- **GetParameters()**: se întorc parametrii spațiului virtual (unul dintre ei foarte important de care va trebui sa se tina cont este viteza de deplasare a utilizatorilor)
- **Move()**: utilizatorul încearcă deplasarea la o anumita poziție a spațiului virtual; i se va da voie sau nu (partea de arbitrare si de logica trebuie sa fie implementata pe server pentru a preveni încercările de trișare ale clienților)
- Etc.

MODULUL CLIENT

Acest modul oferă in principal funcționalitățile de redare 3D si de interacțiune cu utilizatorul.

Modulul client trebuie sa ofere cel puțin următoarele funcționalități către clienții ce accesează spațiul virtual:

- Conectare la spațiul virtual
- Interacțiune cu utilizatorul
    - Deplasarea acestuia in spațiul virtual
    - Comunicare prin chat cu ceilalți utilizatori
- Clientul trebuie sa funcționeze in următoarele 2 moduri:
    - Vizualizare 3D a spațiului virtual:
        - Vizualizare a celorlalți utilizatori din spațiul virtual
        - Vizualizarea evenimentelor din spațiul virtual (deplasare utilizatori, modificare geografie spațiu virtual, etc)
    - Mod consola : in acest mod nu este necesara vizualizarea 3D a spațiului virtual; se vor primi in continuare toate evenimentele din spațiul virtual si se pot afișa informații in consola; in continuare utilizatorul se va putea deplasa in spațiul virtual fie prin deplasarea din taste, FIE prin apelul direct din consola al funcției Move()
    - Etc.

OBSERVATII GENERALE

- Pentru comunicarea in rețea se poate folosi **ORICE** biblioteca de networking se dorește
- Operațiile computaționale ce trebuie efectuate atât de către abordarea CPU cat si cea GPU sunt legate de fizica spațiului virtual si mai exact sunt restrânse la operațiile de collision detection (vezi restricțiile de deplasare prezentate la modulul server)
- Se poate simula un număr mare de clienți folosind modul de consola al aplicației client si posibilitatea folosirii funcției Move() si a răspunsului întors de aceasta sub forma unui script (se pot astfel programa diverse comportamente ale clienților)
- Pentru simplificare se pot reprezenta utilizatorii sub forma unor obiecte simple (de exemplu sfera) si nu folosind avatare complexe ce ar trebui animate (deși ar fi un plus de care se va tine cont suplimentar la evaluare daca ar exista astfel de avatare !!!)
- Este necesara existenta pentru aplicația client si a modulului de vizualizare 3D pentru a vedea ce se întâmpla in spațiul virtual; astfel un comportament de testare ar putea fi : conectarea unui utilizator folosind modul de vizualizare 3D si lansarea conectării a 1000 de utilizatori folosind modul consola
- Se poate folosi pentru redarea / construirea scenei 3D framework-ul de la PPBG

Astfel, Proiectul are următoarele Etape :

- Etapa 0 :
  - Deadline Saptamana 3-4
  - Se va uploada un document care precizează un titlu ales pentru proiect (ex: „WorldOfCollisions” si **numele studentului / echipei** care va lucra la proiect
- Etapa 1 :
  - Deadline Saptamana 7
  - Implementarea Prototipului 1 - 3D Virtual World Physics
  - Upload-ul documentului de etapa care descrie cum au fost implementate funcționalitățile precum si a codului sursa pe SVN
  - **In saptamana 7 se va prezenta la curs etapa precum si funcționarea practica a etapei**
- Etapa 2 :
  - Deadline Saptamana 11
  - Implementarea Prototipului 2 - 3D Virtual World Physics
  - Upload-ul documentului de etapa care descrie cum au fost implementate funcționalitățile precum si a codului sursa pe SVN
  - **In saptamana 11 se va prezenta la curs etapa precum si funcționarea practica a etapei**
- Etapa 3 :
  - Deadline in sesiune (data examen 12 Feb 2027)
  - Implementarea Prototipului 3 - 3D MMO Virtual World Physics
  - Upload-ul documentului de etapa care descrie cum au fost implementate funcționalitățile precum si a codului sursa pe SVN
  - Se va prezenta documentul final precum si funcționarea practica a proiectului

Punctaj proiect :

- Total : 9.5 p
- Etapa 0 : 0.5p
- Etapa 1 : 4p
- Etapa 2 : 3p
- Etapa 3 : 2p

Etapele pot sa fie prezentate la următoarele date :

- Etapa 0 :
  - La data ei de prezentare : punctaj maxim 0.5p
  - La data de prezentare a Etapei 1 : punctaj maxim 0.5p
  - La data de prezentare a Etapei 2 : punctaj maxim 0.5p
  - La data de prezentare a Etapei 3 : punctaj maxim 0.5p
- Etapa 1 :
  - La data ei de prezentare : punctaj maxim 4p
  - La data de prezentare a Etapei 2 : punctaj maxim 3.75p

    - La data de prezentare a Etapei 3 : punctaj maxim 3.5p
- Etapa 2 :
    - La data ei de prezentare : punctaj maxim 3p
    - La data de prezentare a Etapei 3 : punctaj maxim 2.75p
- Etapa 3 :
    - La data ei de prezentare : punctaj maxim 2p